

H-SVM: Hardware-Assisted Secure Virtual Machines under a Vulnerable Hypervisor

Seongwook Jin, Jeongseob Ahn, Jinho Seol, Sanghoon Cha, Jaehyuk Huh, and Seungryoul Maeng

Abstract—With increasing demands on cloud computing, protecting guest virtual machines (VMs) from malicious attackers has become critical to provide secure services. The current cloud security model with software-based virtualization relies on the invulnerability of the software hypervisor and its trustworthy administrator with the root permission. However, compromising the hypervisor with remote attacks or root permission grants the attackers with a full access capability to the memory and context of a guest VM. This paper proposes a HW-based approach to protect guest VMs even under an untrusted hypervisor. With the proposed mechanism, memory isolation is provided by the secure hardware, which is much less vulnerable than the software hypervisor. The proposed mechanism extends the current hardware support for memory virtualization based on nested paging with a small extra hardware cost. The hypervisor can still flexibly allocate physical memory pages to virtual machines for efficient resource management. In addition to the system design for secure virtualization, this paper presents a prototype implementation using system management mode. Although the current system management mode is not intended for security functions and thus limits the performance and complete protection, the prototype implementation proves the feasibility of the proposed design.

Index Terms—Cloud computing, security, virtualization

1 INTRODUCTION

CLOUD computing has emerged as a promising future computing model, which enables elastic resource provisioning to meet unpredictable demands [5], [35]. In cloud computing, users run their programs in virtualized systems, and multiple virtual machines from different users may share the same physical system. However, such cloud computing based on virtualization poses a difficult challenge to securely isolate co-tenants sharing a physical system. The concern for security and privacy protection is one of the most important reasons that have been hindering the rapid adoption of cloud computing [26], [28].

In a virtualized system, a hypervisor, software layer creating and managing virtual machines (VMs), is responsible for isolating any illegal accesses across virtual machine boundaries. However, there have been concerns over vulnerabilities in hypervisors [24], [25]. Although several recent studies improve the security of hypervisors [7], [21], [34], the hypervisors, which have been increasing their code sizes for better performance and more features, have become too complex to simply verify their secure execution [7]. If hypervisors cannot be trusted, even a trustworthy cloud provider cannot guarantee the protection of a virtual machine from a malicious co-tenant.

To support the secure execution of guest virtual machines, and to guarantee the privacy of user information, memory protection across virtual machines is a critical

component. For such memory isolation, current virtualization techniques are based on traditional software and hardware supports for virtual memory. A hypervisor maps the memory of guest VMs to the real memory with guest-physical to machine address translation. Although several different techniques are used to support efficient address translation to the machine address space, all these techniques rely on hypervisors to prevent any illegal memory access across virtual machines [4], [22], [31]. A successful attack on the hypervisor can completely expose the memory contents of guest virtual machines.

In the current memory virtualization techniques, at the highest privilege level, hypervisors can control both aspects of memory virtualization, *memory allocation*, and *memory isolation through address translation*. A hypervisor determines a set of memory pages to be allocated for a VM, and maintains a mapping table from guest-physical to machine address (nested page table (NPT)) for each VM. In this paper, we propose the decoupling of memory isolation from memory allocation, both of which are currently performed by the hypervisor. With decoupling, the role of a hypervisor is limited to the memory resource allocation to utilize the physical memory efficiently. The hardware processor is responsible for updating the page mapping and setting the pointer to the nested page table to schedule a VM on a core. Whenever the nested page table for a VM is changed, the hardware checks whether the update is valid. By this decoupling, Trusted Computing Base (TCB) for memory isolation is reduced to the hardware system from the combination of the hardware and hypervisor in conventional mechanisms. Fig. 1 compares the hardware-based memory isolation with the traditional memory management by hypervisors. The shaded parts represent the TCB for memory isolation for each case.

In this paper, we focus on the protection of guest VMs when the hardware is securely protected in the data center of a cloud provider. With the restricted threat model, our

• The authors are with the Computer Science Department, KAIST, 335 Gwahak-ro (373-1 Guseong-dong), Yuseong-Gu, Daejeon 350-708, Republic of Korea.

E-mail: {swjin, jeongseob, jhseol, shcha, jhuh, maeng}@calab.kaist.ac.kr.

Manuscript received 14 Dec. 2013; revised 23 Sept. 2014; accepted 23 Dec. 2014. Date of publication 8 Jan. 2015; date of current version 10 Sept. 2015.

Recommended for acceptance by C. Xu.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TC.2015.2389792

Authorized licensed use limited to: AJOU UNIVERSITY. Downloaded on September 28, 2023 at 04:35:56 UTC from IEEE Xplore. Restrictions apply.

0018-9340 © 2015 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

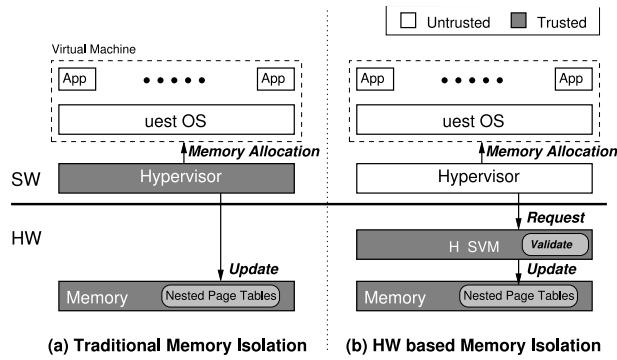


Fig. 1. TCB of hardware-based VM memory isolation.

goal is to minimize necessary changes in the current processor architectures and hypervisors as much as possible. This paper shows that modest extensions on the currently available hardware-assisted virtualization in commercial processors can lead to a significantly improved memory protection under an untrusted hypervisor, as long as the hardware is securely protected in the server room of the cloud provider.

This paper proposes a practical design for the hardware-based VM isolation, called *hardware-assisted secure virtual machine (H-SVM) architecture*, minimizing the changes from the currently available architectural supports for virtualization. Our approach still supports the flexibility of software hypervisors, but the memory protection mechanism is decoupled from the hypervisors, and moved to the hardware processor.

In addition to the H-SVM design, we have implemented a prototype system using System Management Mode (SMM) available in current commercial systems. SMM is designed to provide accesses to systems for remote management. Using SMM, our prototype adds security functions to the SMM layer, which the hypervisor cannot modify or access. However, the current SMM implementation is designed for the management of systems, not for their security, limiting the performance and functionality of our prototype.

The rest of the paper is organized as follows. Section 2 describes the current memory virtualization mechanisms, and the threat model for our study. Section 3 presents the architecture of the proposed H-SVM mechanism. Sections 4 and 5 discuss availability and integrity support by H-SVM. Section 6 presents the prototype implementation with SMM, and Section 7 presents performance evaluation of the prototype. Section 8 discusses prior work, and Section 9 concludes the paper.

2 BACKGROUND

2.1 Hardware-Assisted Virtualization

Memory isolation in the current virtualization techniques is based on the support for virtual memory with hardware address translation and page tables. In processors, a virtual address is translated to a physical address with translation look-aside buffers (TLBs). If the corresponding entry does not exist in the TLBs, either a HW or SW-based page table walker fetches the entry from the page table of the current address space. In this paper, we assume a HW-based page

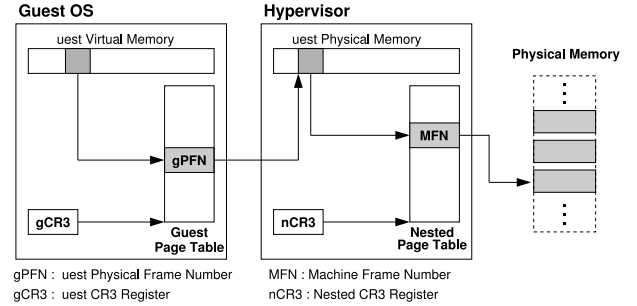


Fig. 2. Address translation with nested paging.

table walker in our architecture, as the proposed architecture aims to move the responsibility of page table management from hypervisors to HW processors. For each context switch between address spaces, the page table register, which points to the top-level page table entry, must be properly set, so that the walker can traverse the correct page table. In the popular x86 architectures, the CR3 register stores the address of the top-level page table.

Virtualization adds an extra translation layer. A virtual address in guest VMs must be translated into a guest-physical address (virtual physical address) like non-virtualized systems, and the guest-physical address is translated to a machine address in the real physical memory. The guest OS maintains the virtual to guest-physical address mapping in per-process page tables, and the hypervisor maintains per-VM guest-physical to machine address mapping tables, called *nested page tables*. When the processor supports only single-page table walks designed for traditional native operating systems, the hypervisor maintains direct translation tables, called shadow page tables, to map virtual addresses to machine addresses directly.

The recent advancement of architectural support for virtualization allows a nested hardware page table walker to traverse both per-process page tables and per-VM nested page tables [4]. Fig. 2 depicts the nested address translation with a per-process guest page table and a per-VM nested page table. Each core has two registers pointing to the two tables, one for the guest page table (gCR3), and the other for the nested page table (nCR3). With the hardware-assisted virtualization, the hypervisor has its own address space, but unlike guest virtual machines, the hypervisor address space uses a single translation without nested paging.

The hardware-assisted virtualization also facilitates a world switch between a VM and the hypervisor contexts. For example, in the AMD-V architecture [2], the context of each VM is defined in a Virtual Machine Control Block (VMCB). The hypervisor, at the host mode, executes the vmrun instruction to switch to a guest VM context. The hardware processor, by executing micro-coded routines, saves the current hypervisor context to a specified area and restores the guest VM context from the VMCB to the processor. The VM context contains the register states including the pointer to the nested page table. If an event, which must be handled by the hypervisor, occurs, the hardware saves the guest VM context in the VMCB, and restores the hypervisor context.

To isolate the memory of each VM, a hypervisor must protect nested page tables from illegal modifications by

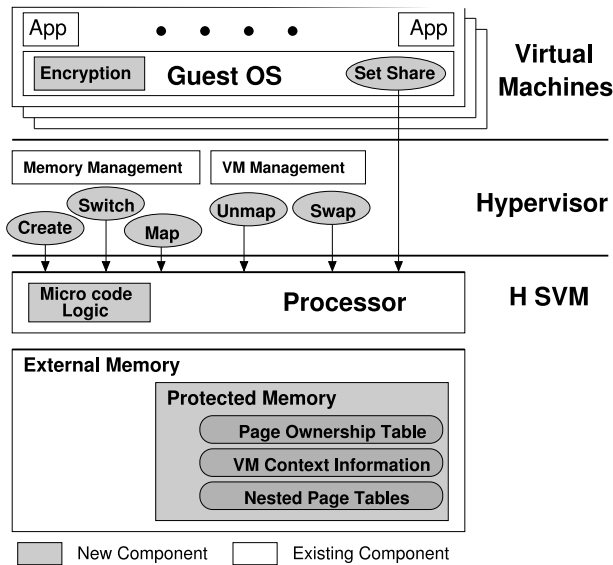


Fig. 3. Hardware-assisted secure virtualization.

guest VMs. Guest VMs cannot read or modify the nested page tables. Also, for each context switch between virtual machines on a core, the hypervisor must change the nested page table pointer directly or by executing the `vmrun` instruction. The hypervisor manages memory allocation by monitoring memory usages of virtual machines, and it can allocate and deallocate pages for a VM. For such mapping changes, the hypervisor can modify the nested page table of the VM. Note that the hypervisor also accesses its own memory space through the address translation mechanism. Since the hypervisor has a complete control over the modification of nested page tables, a compromised hypervisor can read or modify the physical memory assigned for any guest VMs. A malicious user can map physical memory pages already allocated for other VMs to the address spaces of its own VM or to the hypervisor.

2.2 Threat Model

To protect the memory of VMs even under a compromised hypervisor, the proposed mechanism allows only the hardware (H-SVM) to validate and update nested page tables, reducing the trusted computing base to the hardware system. The proposed mechanism can be vulnerable to hardware attacks, such as probing external buses [6] or reading DRAM after power-off [13]. However, we assume that the cloud provider is trustworthy and it does not intentionally attempt to compromise the hardware system. The trustworthy cloud provider protects its servers with physical security measures, as the provider not only has a legal obligation not to access customers' data without explicit permission, but also has a strong business interest in protecting its reputation.

The TCB to protect the memory of guest VMs in the proposed system does not include the hypervisor. We assume that hypervisors are vulnerable to remote attacks by malicious guest virtual machines. The threat model assumes that an adversary with the root permission of the hypervisor may attempt to access the memory of guest VMs. The proposed system can protect the memory of guest VMs as long

as the adversary cannot compromise the physical servers directly. With proper security measures on the server room, getting accesses to the server room, and physically compromising the systems are much harder than getting the root permission by remote attacks.

By not supporting hardware tamper-resistance, we simplify the requirements for H-SVM significantly. H-SVM moves the minimum functionality in traditional hypervisors for updating nested page tables to the hardware processor. The memory protection mechanism in this paper addresses the isolation among virtual machines or from the hypervisor. It does not improve the security of guest operating systems and applications by themselves.

3 ARCHITECTURE

3.1 Overview

H-SVM improves memory isolation among VMs by blocking direct modifications of nested page tables by a hypervisor. Nested page tables for VMs are stored in the protected memory region, which can be accessible only by the H-SVM hardware. For any changes in memory allocation for a VM, the hypervisor, at the privileged level, makes a request to H-SVM to update the nested page table for the VM. If the hypervisor is compromised, it can try to allocate a physical memory page already assigned to a VM to the address space of the hypervisor or a malicious VM. Before updating the nested page table, H-SVM checks whether the request may violate memory isolation among VMs. If a physical memory page is deallocated from a VM, H-SVM cleans up the deallocated page by setting all the bytes to zeros.

Fig. 3 presents the overall architecture of H-SVM. H-SVM can be implemented either as a separate controller in the processor chip, or can be added as microcode routines. Such microcode routines are commonly used to implement complex features in the x86 architecture. In the rest of the paper, we will assume such a microcode implementation. The nested page tables of all VMs are stored in protected memory pages. The protected memory region is just part of the physical memory, which is accessible only by H-SVM. H-SVM blocks accesses to the protected memory even from the hypervisor, by denying page mapping requests to the protected pages.

H-SVM maintains several data structures, including VM context information, nested page tables, and a *page ownership table*, in the protected memory region. The VM context information contains various states such as the address of the top-level nested page table, and an encryption key created for the VM. The VM context information is similar to the VMCB in the AMD-V architecture. The page ownership table tracks the owner of each physical memory page, and thus the number of entries is as large as the number of physical memory pages in the system. Each entry, corresponding to a physical page, records the ownership of the page. A VM, hypervisor, or H-SVM itself can be the owner of a page. If H-SVM is the owner of a page, the page is used for the protected memory area. The page ownership table is used to verify whether a page map request from the hypervisor is valid or not.

When the control is transferred to the hypervisor by interrupts, H-SVM must save the register states of the

current virtual CPU (vCPU) to the VM context information. After saving the VM state, H-SVM sets the page table pointer to the page table used by the hypervisor. When the hypervisor schedules a vCPU to a physical core, the hypervisor requests H-SVM, by executing a privileged instruction, to place a VM to a core. For the scheduling request, H-SVM restores the VM state from the VM context information, including the nested page table pointer. As discussed in Section 6.1, the current x86 processors already support a similar world switch operation, such as the *vmrun* instruction in AMD-V. A main difference of H-SVM from the current support for *vmrun* is that H-SVM requires that the VM context information must not be accessible by the hypervisor.

With H-SVM, the hypervisor conducts normal memory management operations for VMs. For a VM creation, it decides a set of memory pages for the new VM, and makes requests to update the nested page table of the newly created VM. The hypervisor can also deallocate memory pages from a VM, often by the ballooning technique, but the actual updates of nested page tables occur in H-SVM. The role of H-SVM is limited only to the protected update of nested page tables and validation before any modification to nested page tables. Hypervisors still have a control over the memory resource management to assign memory pages to VMs.

3.2 H-SVM Interfaces and Implementation

This section describes the basic interfaces of H-SVM. Hypervisors or VMs execute special instructions to make requests to H-SVM. There are four basic interfaces to initialize the VM context information, to update nested page tables, and to schedule a VM.

Create VM. When a hypervisor creates a VM, it requests H-SVM to create a new nested page table for the VM. H-SVM initializes the VM context information, and creates a nested page table for the VM. After the data structures are created in the protected memory area, H-SVM returns a VM identifier, which the hypervisor will use to designate the created VM for subsequent interactions with H-SVM. H-SVM also creates a per-VM encryption key, which will be used for page swap requested by the hypervisor.

Delete VM. When a hypervisor destroys a VM, it requests H-SVM to delete existing nested page table for the VM. H-SVM also destroys VM context information. H-SVM clears memory of the VM before destroying the nested page table. It guarantees the confidentiality of VM. H-SVM resets the entry of page ownership table because of allocating the memory to other VM. Finally, H-SVM clears the contents of VM like the per-VM encryption key and VM identifier.

Page map. To assign a physical memory page to a VM, the hypervisor requests a *page map* operation to H-SVM. A *page map* operation maps a machine memory page (frame) to a guest-physical page by updating a nested page table entry. The critical component for memory isolation is to check the ownership of a physical page for each *page map* operation by H-SVM. Before updating the nested page table entry, H-SVM must check, by looking up the page ownership table, whether the physical page is owned by another VM. If another VM already owns the requested physical page, the *map* operation is aborted. When a nested page table

entry is updated for a VM by the request of the hypervisor, the VM becomes the owner of the physical page. This checking mechanism prevents a compromised hypervisor from creating any illegal mapping to the pages already used by other VMs.

Page unmap. To deallocate a physical memory page from a VM, the hypervisor makes a *page unmap* request to H-SVM. H-SVM modifies the corresponding nested page table entry, and clears the content of the memory page before completing the operation. H-SVM also resets the owner of the page in the page ownership table, marking it as a free page. With clearing, the contents of free pages cannot contain any information from prior guest VMs.

Context save. When the hypervisor needs to schedule a VM to a core, the VM context information including register states must be saved and restored. The context information contains the page table pointer to the nested page table and register states. So H-SVM must protect the context information from the hypervisor. The hypervisor requests *context save* to H-SVM with a VM identifier. Then H-SVM saves register states of the core running the VM into the protected memory. So hypervisor cannot touch any context information of the VM. This operation is similar to *vmexit* in AMD-V.

Context restore. When a VM is scheduled in, hypervisor requests *context restore* to H-SVM. H-SVM loads the VM context information into the core to set up the register states. As only H-SVM can update the nested page table pointer and register states, the hypervisor cannot force a running VM to use a compromised nested page table. This operation is similar to *vmrun* in AMD-V.

3.3 System-Level Support

In addition to the basic functions H-SVM must support, there are several system-level requirements to consider, including page sharing, page swapping, and protection from DMA. In this section, we address such system-level requirements for H-SVM.

3.3.1 Page Sharing

Some physical pages can be shared between a guest VM and the hypervisor, or among guest VMs. Such page sharing is used for communication for I/O requests, or content-based memory sharing [12], [20]. By default, H-SVM allows a physical memory page to be mapped only to a single VM. However, to make a page sharable, H-SVM needs an explicit permission from the guest VM. By requiring the permission from the guest OS, the guest OS can be prepared not to store any sensitive data on such shared pages.

3.3.2 Page Swapping

In virtualized systems, both guest OSes and the hypervisor can swap out memory pages to disks to use the limited physical memory efficiently. For a given guest-physical memory capacity, a guest OS manages the memory with traditional OS-level memory management policies. However, in virtualized systems, the memory allocated for each VM can change dynamically. One of the most common techniques for dynamic VM memory management is a *ballooning* technique [33]. With ballooning, the guest OS decides which memory pages are rarely used, and deallocates the pages by

assigning them to the balloon driver space. The ballooning technique can provide more efficient memory management than direct swapping by the hypervisor.

For these page swap operations by a guest OS, the guest OS is responsible for encrypting the contents, and maintaining the integrity of swapped pages. Since the guest OS needs to protect its other files in addition to the swap file, it must use a secure file system, which can guarantee the confidentiality and integrity of its data in untrusted storage.

In addition to ballooning, the hypervisor may also evict memory pages to its disk swap space directly. However, the direct page swaps by the hypervisor are much less commonly used than ballooning [33]. Although its uses are less common than ballooning, H-SVM can support direct page swapping by the hypervisor. As the hypervisor writes the memory pages to the swap disk, the contents of evicted memory pages must be encrypted by H-SVM before being accessible to the hypervisor. When a VM is created, H-SVM creates an encryption key for the VM. When the hypervisor picks a victim page, H-SVM encrypts the page with a per-VM key, and allows the hypervisor to read the page. Per-VM keys are stored in the protected memory region. The integrity issue of swapped pages will be discussed in Section 6.3 in details.

3.3.3 Protection from DMA

Securing nested address translation protects the memory of guest VMs from accesses through instruction executions in processors. The other source of memory accesses in systems is Direct Memory Access (DMA) for I/O handling. As the hypervisor or the domain0 VM controls I/O devices and DMA mechanisms, a compromised hypervisor can potentially read or corrupt the guest VM memory through DMA. H-SVM protects the memory of guest VMs from DMA, by extending IOMMU (I/O memory management unit), which is supported by both Intel VT-d and AMD-V [1], [14]. IOMMU supports an address translation mechanism from a device address space to the physical memory space. The address range for a DMA access from I/O devices must be translated to physical addresses with an I/O page table. Currently, the hypervisor manages and updates the I/O page table for each VM. During the address translation process, IOMMU enforces memory protection, by checking whether DMA accesses are allowed or not for the physical address range.

As DMA accesses are processed with an address translation mechanism using page tables, similar to memory accesses from CPUs, the same mechanism of protecting nested page tables is used to protect I/O page tables. Unlike the current IOMMU architecture, I/O page tables should reside in the protected memory region along with nested page tables, and only the H-SVM is allowed to update the I/O page tables directly. Guest VMs or the hypervisor can request H-SVM to update the DMA protection status of memory pages owned by them. H-SVM must allow the change of the protection status, only when the requesting VM or hypervisor owns the corresponding page by checking the page ownership table.

An alternative way of securing guest VM memory from DMA is to extend Device Exclusion Vector (DEV) [3]. DEV

is a bit vector representing the entire physical memory at page granularity [3]. For each DMA operation, the hardware system checks whether the physical address range for the operation is allowed by checking the corresponding bits in DEV. Unlike IOMMU, DEV does not support address translation, but it can enforce memory protection for DMA. Currently, the hypervisor can change the content of DEV, and thus control whether DMA is allowed for each physical memory page. To protect guest memory from accesses through DMA, DEV must be moved to the protected memory region controlled by H-SVM.

3.4 Cloud-Level Support

One of the key issues to secure guest VMs with H-SVM is to verify the integrity of newly launched guest VMs. A malicious hypervisor can use a wrong VM image to start a new VM, and provide the compromised VM to its user. The cloud architecture must support the integrity checking mechanism for a VM before it can become accessible to its user. To check the VM image, H-SVM has a mechanism to verify the hash of VM image securely without intervention of the hypervisor. Furthermore, a unique pair of private and public keys is assigned to each H-SVM hardware for secure communication with a cloud managing system either within or beyond the cloud provider boundary. The details of VM integrity checking mechanism and secure communication supports are discussed in our prior study on H-SVM [15].

3.5 Limitation for Content-Based Page Sharing

A special type of page sharing used in virtualized systems is *content-based page sharing*. For such content-based page sharing, the hypervisor of a system checks whether the contents of different memory pages from different VMs are equal, and allows read-only sharing of the pages with the same content. If a VM attempts to update a content-shared page, the hypervisor allocates another page for the VM, to protect the original page used by other VMs. Such a copy-on-write (COW) mechanism isolates memory pages used by different VMs, by duplicating content-shared pages once the pages are about to be updated. Content-based page sharing (content-sharing) reduces unnecessary duplications of memory pages in physical memory, and thus improves the efficiency of memory utilization.

However, guest VMs are not aware of the content-based sharing of memory pages. The hypervisor actively searches pages with the same content, and maps equivalent virtual pages of guest VMs to the same physical page. The COW mechanism guarantees the protection of memory pages even though the same physical pages are assigned to different VMs. The hypervisor is responsible for guaranteeing the isolation of VM memory for content-sharing. Therefore, a compromised hypervisor can exploit the content-based sharing, and make VMs use incorrect memory pages.

A straightforward security solution for content-based page sharing is to disable it. In public cloud services, such content-based sharing may not be commonly used yet. When the memory size is mandated by SLA (Service-Level Agreement), the cloud provider does not employ the content-based sharing scheme. However, in highly consolidated

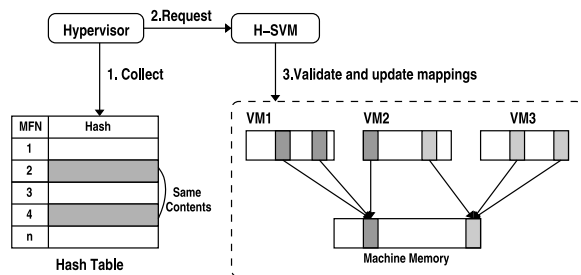


Fig. 4. Contents-based page sharing with H-SVM.

virtualized systems with many active virtual machines, the overall memory usage is critical for performance.

However, securely supporting content-based sharing with H-SVM is possible only by limiting its sharing efficiency. To support content-based sharing, H-SVM must help the hypervisor find pages with the same content, since the hypervisor cannot directly read the memory of guest VMs. H-SVM must also verify whether two pages have the same content as the hypervisor requested, and update the nested page tables of the corresponding VMs. Three extra operations described in Fig. 4 are necessary in H-SVM to support such content-based sharing. First, H-SVM must provide the hypervisor with a new interface to return a hashed value of a memory page as a compact summary of the page. Second, for sharing requests from the hypervisor, H-SVM must validate whether the two pages actually have the same content. Third, a shared page should be marked as a read-only page, and when a VM attempts to update the page, H-SVM must invoke a fault to be handled by the hypervisor.

However, a key security problem with the aforementioned scheme is that the exposure of the hash value of each page can indirectly reveal the content of pages. The hypervisor may infer the hash function by comparing the hash value and a known page content, although inferring a hash function can be a significantly time consuming procedure. Furthermore, identifying pages with the same content based on hash values, can also be a useful information for the malicious user. Due to the exposure of hash values for pages, the content-based sharing support has an inherent security vulnerability.

With the security vulnerability of exposing the hashed values of page to the hypervisor, there can be several possible mitigating solutions, although the solutions have their own limitations. First, the guest OS can request H-SVM to limit the range of memory pages for content-based sharing. Second, H-SVM can use a different seed for hashing for each VM, which disables sharing across VMs, but allows only intra-VM sharing. Third, the resolution of hash values can be reduced to incur more hash conflicts. Although it reduces the efficiency of hash-based search with many false positive, the content inference from hash values becomes less valuable to the malicious hypervisor.

4 INTEGRITY

The last aspect of security is whether H-SVM can protect the integrity of VMs even under a malicious hypervisor. The integrity of a VM becomes vulnerable when its state is stored in the system area controlled by the hypervisor. For example, swapped files from a VM is stored in disks where

H-SVM cannot directly control. Although the confidentiality can be guaranteed by encryption, the integrity of the stored states must be protected by an additional mechanism. Without proper integrity support, the malicious hypervisor can arbitrarily change the states in swapped pages or during migration. In this section, we discuss how H-SVM can support integrity for system BIOS, swapped pages, and VM images.

However, the integrity support of H-SVM does not protect the integrity of memory pages completely. By remapping attacks, the hypervisor can assign zero pages for VMs. This section discusses the integrity problem.

4.1 System BIOS

For the effectiveness of H-SVM, the integrity of system BIOS must be guaranteed. System BIOS contains the critical setting information, which turns on or off H-SVM and sets the addressing mode. Furthermore, BIOS for x86 architectures also contains patches for microcodes used in processors, with which the routines of H-SVM are implemented. H-SVM may use a conventional mechanism using TPM (Trusted Platform Module) to validate the content of BIOS [30]. With the TPM support, the content of system BIOS is securely summarized and stored in the protected hardware storage. The software system including the hypervisor cannot update the stored information. The cloud manager must remote-attest the BIOS content using TPM-based remote attestation protocols. Through the attestation process, the cloud manager uses only the physical system with an intact system BIOS.

4.2 Swapped Pages

As discussed in Section 3.3, the hypervisor can swap out guest memory pages on memory pressure. Although encryption can guarantee the confidentiality of memory pages, encryption alone does not support the integrity of swapped-out memory pages, since the hypervisor can change the swap files in disks arbitrarily. To maintain the integrity of swapped pages, H-SVM also must maintain the hashes of swapped-out pages in the protected memory region. When a page is swapped in, the page is first decrypted, and its hash is calculated. The hash value of the swap-in page must be compared to the stored value for integrity checking. However, maintaining such hash values of memory pages causes memory overheads in the protected memory and computation overheads for H-SVM routines. In this paper, we discuss two different organizations of hash values for swap pages, with different computation complexities and memory overheads.

Hash array. The simplest organization of hash values of swap memory pages is to maintain a full sequential hash array for the entire physical memory space. This simple method provides a quick update of hash value, since one hash entry needs to be updated for each swap-out page and the index can be quickly calculated. For a swap-in, the hash value in an entry of the array is compared to the swapped-in page. However, the critical disadvantage of this approach is the memory overheads. If a system has 8 GB machine memory and the page size is 4 KB, 64 MB memory must be used for storing hash values in the protected memory region.

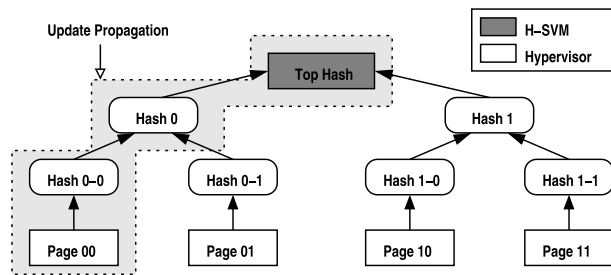


Fig. 5. Hash tree.

Hash tree. The second organization for hash values for swap pages is to maintain per-VM hash trees as described in Fig. 5. A hash tree is a data structure guaranteeing integrity, which is mainly used for verifying transmitted data between remote machines in peer to peer networks [19]. For H-SVM, tree leaves contain hash values of swapped pages, and each parent node contains the summary hash value from its child nodes. The top hash (root hash) represents the summary hash value of all the swap pages.

Each VM has a hash tree for its swapped pages, and the hypervisor maintains the per-VM hash trees. However, H-SVM stores only the top hash of each tree in the protected memory region. For a page swap request, the hypervisor must provide H-SVM with the hash values of the associated branch in the hash tree in a memory page. H-SVM will verify the provided hash tree by comparing the computed value with the stored top hash, and update the top hash with the hash of the new swap-out page. For a swap-in, the hypervisor must provide H-SVM with the hash values of the associated branch from the hash tree. H-SVM decrypts the page first, and its hash value is computed and verified with the branch.

A possible vulnerability of supporting page swap by the hypervisor is that the consistency of pages can be violated. The malicious hypervisor can request a swap operation for a page while the page is being updated by the owning VM in another core. The swapped page may not have the latest updates. To prevent such integrity violation, H-SVM must block the execution of virtual cores for the VM during the swap operation. Although such blocking can prevent the consistency problem, it will reduce the performance of the VM significantly. Considering the page swap by the hypervisor is used as the last resort to fix memory thrashing in the system, such performance degradation can be tolerable, as VMs in the system are already suffering from memory thrashing.

4.3 VM Images for Checkpoint and Migration

The state of a VM becomes vulnerable for integrity violation when it is not directly protected by H-SVM, as discussed in the previous section for swap files on the system disks. Another case for the integrity vulnerability for a VM is the VM image in checkpoints stored locally or the memory and CPU states during VM migration. During the checkpointing or migration procedure described in Section 4.3, the hash tree is constructed for the VM image. H-SVM must generate the top hash value for the VM image, and sign the value to protect it from forgery by malicious hypervisors.

4.4 Vulnerability for Remapping Attack

One of the limitations of H-SVM for memory integrity is that the hypervisor can assign a zero page to a VM.

Although zeroing the content of a page during unmap operations, provides the confidentiality of pages, the hypervisor can unmap a page silently, and then assign a new page for the same guest-physical page. The execution of the VM can be affected by the change of page content to zero contents. Preventing such attacks may require to limit the dynamic memory management by the hypervisor. H-SVM may block the unmapping of pages by the hypervisor, unless the guest OS allows the unmap operation through memory ballooning techniques. With memory ballooning, the pages can be unmapped only after the guest OS releases the pages. Completely supporting the integrity protection for mapped pages will require the restriction of dynamic memory management by the hypervisor after a VM is created.

5 AVAILABILITY

In this section, we describe how to support the availability of virtual machines with H-SVM, even when the hypervisor is malicious. For perfect security, guest VMs must function normally even under a malicious hypervisor. We discuss why H-SVM cannot directly avoid the violation of availability, and present solutions for the availability problem. Furthermore, we extend the availability of guest VMs in clouds from minimum functional availability to guaranteeing the Service-Level Agreement.

5.1 Detecting Availability Violation

For secure cloud services, the availability of guest VMs, even under a compromised hypervisor, must be supported. Applications running on guest VMs may conduct critical services for their users, and suspensions of such services may cause a significant harm to the users' interests. In this section, we discuss how to detect the violation of availability or Service-Level Agreement with H-SVM, even though the hypervisor is compromised. H-SVM does not directly guarantee the availability of guest VMs. To allow complex but effective resource management by a hypervisor, the hypervisor still controls the resource allocation policies for guest VMs even with H-SVM. By allowing such a flexible management, a compromised hypervisor may not schedule guest VMs to CPUs, deallocate memory, or drop packets for VMs. Guest VMs may become unavailable, or their performance may decrease.

Although H-SVM cannot directly enforce the availability of guest VMs, it can still detect the violation of availability and report such a violation to the cloud manager to notify the administrator about the problematic hypervisor. Upon receiving such availability violation notices, the cloud manager may securely migrate the VM to a different system, or request administrators to intervene manually to fix the problem. In this section, we propose two levels of such a detection mechanism to report availability violation, minimum functional availability and fine-grained assessment of SLA.

The first level of availability support checks whether a guest VM is functioning by receiving heartbeats from the guest VM regularly. The cloud manager monitors the status of each guest VM by periodically sending a message to the VM, and waiting for a response. This simple approach guarantees the minimum availability, but cannot detect possible performance degradations caused by a malicious hypervisor.

For example, a compromised hypervisor may reduce CPU time slices for guest VMs, but periodic heartbeats cannot detect such a problem. A compromised hypervisor can also deallocate memory pages for guest VMs, causing excessive page swaps. To detect such performance degradations, it is necessary to detect whether a hypervisor allocates resources promised by SLA to guest VMs.

5.2 Supporting Service-Level Agreement (SLA)

The second level of availability support verifies the allocation of CPU and memory resources for fine-grained validation of resource allocation. In current cloud services, each user VM is guaranteed to receive a certain amount of CPU share and memory by SLA. However, a compromised hypervisor can reduce the promised resources from guest VMs to use the resources for malicious VMs.

H-SVM can be extended to record CPU and memory allocation status securely with hardware. With H-SVM, the hypervisor can allocate CPU shares and memory pages only through H-SVM, and thus H-SVM can always securely verify how much CPU share or memory is assigned for each VM. If a compromised hypervisor does not allocate CPU shares or memory pages for a VM, H-SVM can detect it. As shown in Section 4, H-SVM with a secure communication protocol to the cloud manager, can send the resource allocation status of a VM to the cloud manager even if the hypervisor in the middle is malicious.

CPU allocation. For CPU scheduling, H-SVM can record the cycle count between the execution of `schedule VM` by the hypervisor and the exit of guest VM run, due to the expire of time slice (interrupt) or VM exit events. VM exit events include the execution of privileged instructions which can only be handled by the hypervisor, and the execution of `halt` instruction to relinquish CPUs voluntarily by guest VMs. For I/O intensive or interactive workloads, guest VMs often relinquish CPUs while they are waiting for I/O events. Such voluntary relinquishment of CPU cycles complicates the accurate assessment of CPU allocation by H-SVM.

H-SVM needs to verify whether the hypervisor is providing each VM with the CPU share promised by SLA. H-SVM at the same time must consider that each VM may relinquish unused CPU cycles. Therefore, for a fixed time period, it checks how many times a VM is scheduled to a physical CPU, and how long the VM runs until the VM is interrupted by the expiration of time slice. H-SVM accumulates the CPU usage information, which should be collected by the cloud manager to verify the availability of each VM.

Memory allocation. For memory allocation, H-SVM can trace the total memory pages allocated for a VM, as it executes `page map` and `page unmap` operations. The total memory pages assigned to each VM is increased or decreased for the two operations. Since memory pages can be added or removed to a VM only by H-SVM, H-SVM can securely measure the total physical memory size allocated for a VM.

The cloud manager can collect the CPU and memory allocation information by periodically requesting it from the hypervisor of each system. The hypervisor will request H-SVM to provide the information, and H-SVM will securely sign it before returning to the hypervisor to prevent forgery.

Checking I/O performance. As H-SVM cannot be involved with I/O operations and verifies only CPU and memory allocation, I/O performance must be sampled by a monitoring tool running in guest VMs. Such a monitoring application may periodically sample the performance of each I/O device to validate the I/O scheduling policy by the hypervisor. The monitor application on each VM will report the measured I/O performance to the cloud manager. As a malicious hypervisor cannot affect the user application in a guest VM with H-SVM, the monitoring application can securely measure the I/O performance.

6 PROTOTYPE IMPLEMENTATION

A complete implementation of H-SVM requires several changes, although relatively minor, of processor architectures on top of the current hardware support for virtualization. In this section, we present our prototype design using System Management Mode, which is a software execution mode for management isolated from the hypervisor. The prototype design is focused on the memory protection mechanism of H-SVM, and cannot protect CPU register states completely due to the current limitation of SMM. Although the prototype is much less efficient than the proposed H-SVM design and cannot support some security critical components of the complete H-SVM design, the purpose of this implementation is to prove the feasibility of H-SVM before its full implementation in future processor architectures.

6.1 System Management Mode

System Management Mode is a special mode supported by x86 architecture, mainly used for maintenance functions in computing machines. An x86 processor can enter into SMM by SMI (system management interrupt), which is a physical hardware interrupts invoked by abnormal system events such as a high temperature or system error, in addition to SW invocation of the interrupt. The SMI handler is loaded into SMRAM (System Management RAM) from BIOS when the system boots. SMRAM is a part of main memory but accessible by only the SMI handler, which is similar to the protected memory region in H-SVM. When an SMI event occurs to a processor, the processor saves all of the running task contexts in the SMRAM area. During the SMI handling, exceptions and interrupts that can cause control transfer to the privileged software, are disabled. Disabling the control transfer prevents the OS or hypervisor from interrupting the SMI handling, and thus affecting the SMM execution. When the processor exits from SMM, the processor restores the context saved in SMRAM to the processor registers, resuming normal execution. The processor in SMM can access main memory and hardware devices with I/O port with full permission, including the memory area in which hypervisor or kernel is loaded.

The SMM mechanism provides an isolated execution environment even from the hypervisor, since the SMI handler is loaded in the protected memory region, and the execution during SMM cannot be interrupted by the hypervisor. Furthermore, during the SMM state, any memory or HW devices are accessible without limitation. We use SMM to execute H-SVM operations by using a custom SMI handler. In our prototype, we modified the system BIOS to

load our prototype SMI handler, which provides a limited functionality of H-SVM. In the rest of this paper, H-SVM_{smm} denotes our SMM-based prototype instance of H-SVM.

6.2 SMM Initialization

Initialization of H-SVM_{smm} requires several steps to allocate a protected memory region, and set up communication channels to the hypervisor.

Protecting the SMI handler. The first step in initializing H-SVM_{smm} is to ensure the protection of the SMI handler from the untrusted hypervisor with the highest privilege level. In the current SMM design, BIOS can lock the SMI handler by setting a bit in the MSR register, and even the hypervisor cannot modify the SMI handler.

Setting SMRAM. The protected SMRAM is accessible only during the SMM execution, and the memory region must be set during the initialization. The modified BIOS allocates 64 MB memory to SMRAM, by updating E820 System Address Map to mark the SMRAM region as reserved area. This process prevents the hypervisor from accessing the SMRAM region.

Communication channel. The hypervisor must invoke memory map and unmap interfaces in H-SVM_{smm} to manage the VM memory. To allow such communication for invoking H-SVM interfaces, BIOS allocates 4 MB memory for the communication channel to pass arguments to H-SVM_{smm}.

Loading the SMI handler. BIOS loads the SMI handler into SMRAM. The SMI handler entry point is located at the address 0x8000 of SMRAM. When an SMI occurs, the processor jumps to the SMI handler entry points. In SMM, address translation is disabled before initialization. During the initialization, the SMI handler enables segmentation and constructs page tables to enter long mode. When an SMI is invoked, all of the cores enter the SMM automatically, which is a performance limitation of the current SMM implementation. We will discuss the limitation of SMM in Section 6.5.

6.3 Shadow Nested Page Table

For each VM, its nested page table defines the machine memory allocated for the VM. For security of the VM, NPT must be located in SMRAM accessible only by the SMI handler. The current SMM implementation blocks any accesses, even read-only accesses to SMRAM from the hypervisor. However, the hypervisor also needs to read the nested page table to manage the VM. To allow such limited and verified accesses for the nested page table, H-SVM_{smm} supports shadow data structures for NPT.

H-SVM_{smm} maintains shadow copies of nested page tables in the hypervisor memory regions, so that the hypervisor can read the data for managing VMs without affecting security-critical components. The shadow data structures are also used for sending request arguments from the hypervisor to H-SVM_{smm}. To change a page mapping, the hypervisor updates a shadow nested page table, and invoke the SMI handler for H-SVM_{smm}. The SMI handler verifies the change, and updates the original nested page table only for valid requests.

Fig. 6 depicts the locations of nested page tables and shadow tables. Real nested page tables are located in

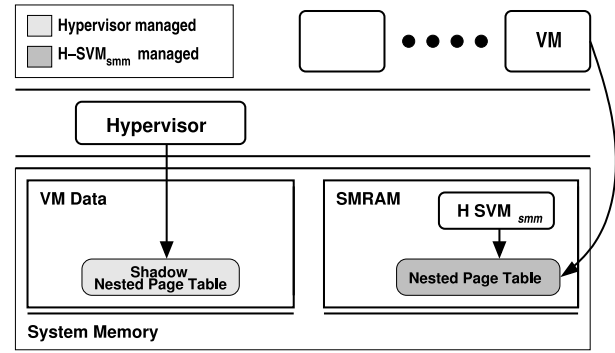


Fig. 6. Shadow nested page table.

SMRAM, but shadow nested page tables are in the normal hypervisor memory. To synchronize between two nested page tables, the hypervisor first updates the shadow nested page table to alter the memory mapping of a VM. Then hypervisor calls the SMI handler to request the same memory mapping changes. Even though hypervisor maliciously changes the memory mapping in the shadow nested page table, H-SVM_{smm} verifies the request and rejects malicious ones from the hypervisor.

6.4 SMM Operations

In this section, we describe SMM operations corresponding to the H-SVM interfaces. To call an SMM operation, the hypervisor initiates a system management interrupt with a call ID and arguments in the communication channel. To prevent concurrent invocations of the SMI handler from multiple cores, the hypervisor must acquire a lock before entering the critical section. After processing the SMI handler, H-SVM_{smm} releases the lock.

Create VM. The SMI handler for H-SVM_{smm} creates the root page of a nested page table for the requested new VM, and VM contexts for VCPUs with the pointer to the nested page table. In addition to two critical data structures, the SMI handler creates an extra red-black tree to track memory allocation information for efficient allocation and deallocation of VM memory in the SMI handler. The tree is updated during page map and unmap operations, and during a VM destruction, the SMI handler traverses the tree to reclaim all memory pages.

Delete VM. The SMI handler deallocates memory space for the nested page table, and VM context information. By using the aforementioned red-black tree, the SMI handler identifies which VM memory pages are allocated. These memory pages are deleted and cleaned by the SMI handler for confidentiality. When SMI handler deletes memory pages, the ownership table is also updated to mark as a freed page.

Page map. To assign a physical memory page to a VM, the SMI handler updates the appropriate entry of the nested page table. Prior to a nested page table update, the SMI handler checks page ownership. If the memory page is already assigned to another VM, the SMI handler rejects the page map request. Otherwise, the SMI handler sets the ownership table and updates the nested page table. The SMI handler also inserts the new memory page to the red-black tree for fast tracking.

Page unmap. To deallocate a physical memory page from a VM, the SMI handler updates the appropriate entry of the

nested page table as invalid. Before updating the page table entry, the SMI handler cleans the memory page. The SMI handler also deletes the corresponding entry of the red-black tree and reset the entry in the page ownership table.

Get instruction. If a VM executes a privileged instruction, it will trigger VMEXIT. The current hypervisors must read the instruction and operands from the guest VM memory directly to emulate. Since H-SVM disables such direct memory accesses, H-SVM must change the interface for the privileged instruction execution. The interface must be modified not to allow the hypervisor to read the instruction directly. For each VM exit, H-SVM must identify why the VM exit occurs based on the VMCB information containing EXITREASON and EXITINFO fields. Based on the information, H-SVM must access the VM memory to read the privileged instruction and operands for the hypervisor, and deliver them to the hypervisor through the get instruction executed by the hypervisor. However, the current prototype is not complete for the support.

This prototype version of H-SVM_{smm} requires a modest change of the hypervisor. For VM create, VM delete, page map, and page unmap operations, the hypervisor must call the SMI handler to process the operations. We use Xen 4.0.1 as the hypervisor. The changes to the hypervisor are minimal with less than 500 lines of codes.

6.5 SMM Limitation

The current SMM is designed to support system management task during urgent system events such as high temperatures or system errors. Due to the management purpose of SMM, it has several limitations both in performance and security, if security functions use SMM as an isolated layer under the hypervisor. Although we chose to use SMM to show the feasibility of the H-SVM design, our SMM-based realization, H-SVM_{smm}, also has several limitations. In this section, we discuss the limitation of H-SVM_{smm} due to the SMM restriction. This discussion can lead to a future extension of SMM, incorporating security requirements in addition to the current system management requirements. The first two limitations cause performance degradation, but the next two limitations cause incomplete security protection by H-SVM_{smm}.

4 GB address space limit. In SMM, the address space is limited to 4 GB. However, during the page unmap operation, the SMI handler must clean the free page by filling it with zero content, but the SMI handler cannot directly write the page with a higher address than 4 GB limit. To implement the page sanitization, the SMI handler disables any interrupts for a core, and invokes the cleaning code in the normal mode after exiting SMM. By disabling any interrupts, the hypervisor or any other VM cannot affect the sanitization code execution even in the normal mode. Furthermore, to protect the cleanup code, the cleanup code is initially stored in SMRAM, and only after interrupts are disabled, the code becomes exposed to the normal mode shortly, by decreasing SMRAM region. After the page cleanup is completed, the control is returned to the SMI handler, increasing SMRAM to protect the code and enabling interrupts back.

All core SMM invocation. When an SMI occurs, all of the cores in a system must enter SMM. However, since H-SVM_{smm} needs only one core to run the SMI handler, the

rest of cores do not need to be in SMM. The current implementation of SMM with such synchronized SMM invocation reduces the overall performance of H-SVM_{smm}. To mitigate the performance impact, the rest of the cores exit SMM immediately.

When a core run still in SMM, other cores can attempt to invoke H-SVM_{smm}. In our prototype, a global spin lock is used for serializing such overlapping H-SVM_{smm} requests. Although a global spin lock is used for the simplified design, the execution of critical operations by the lock holder is not preempted to avoid any lock holder preemption problems.

Lack of protection for register states. As a prototype, H-SVM_{smm} is focused on protecting the memory state, but the register states are not completely protected, due to the limitation of the current VM context switching mechanism. Although the virtualization HW support saves and restores a subset of register states, the hypervisor still needs to save and restore the rest of register contexts. Due to the limitation, the hypervisor can read and update the register contexts, creating a security hole. However, this problem can be solved by extending the HW support for context save and restore operations for the entire register states in future architectures.

Unprotected nested page table pointer. Although the hypervisor invokes the SMI handler to request security critical functions, a certain security-sensitive state can still be accessed by the hypervisor directly. One of the critical problems is the accesses to nCR3, which is a nested page table pointer. In the current SMM and virtualization support, the hypervisor can update the content of nCR3. With the capability, the hypervisor can set up a decoy nested page table, and force a VM to use the decoy nested page table. To provide a completely secure memory protection, any update to nCR3 must be allowed only in SMM, by causing SMI in any access attempt to nCR3. An alternative way is to check nCR3 in SMM whenever an H-SVM operation is invoked. This delayed checking can reduce the chance for the hypervisor fakes the nested page table, although the system is not completely secure.

Due the last two limitations, the current prototype provides a limited security function. However, designing H-SVM_{smm} with the SMI handler provides the understanding of the design complexity of H-SVM. In this next section, we present the performance overheads and complexity of H-SVM_{smm}.

7 PERFORMANCE EVALUATION

7.1 Methodology

All experiments run on a six core 2.8 GHz AMD Phenom x6 machine with 8 GB DDR3 memory. The Phenom CPU supports hardware-assisted virtualization including support for nested page tables. We use Xen 4.0.1 as the hypervisor, and Ubuntu 10.04 runs on each virtual machine as the guest operating system. We run the Xen hypervisor with the hardware-assisted full virtualization mode with H-SVM_{smm}. We modified the current BIOS to add the customized SMI handler which implements H-SVM_{smm}. In our prototype, 64 MB SMRAM is used for storing nested page tables and other security data. Most of SMRAM is used for nested page

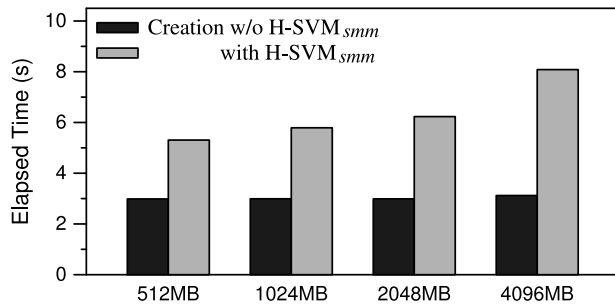


Fig. 7. Elapsed times for VM creation.

tables. In the worst case that all VMs consist of the smallest 4 KB pages, a nested page table uses 2 MB SMRAM per 1 GB VM memory.

In virtualized systems, page map operations occur intensively when a VM is created, to allocate the initial memory for the VM. To evaluate the performance impact of extra steps of page map, we use a micro-benchmark which creates VMs with various sizes of memory. As a VM has more memory, more page map operations must occur.

To show the effect of realistic applications, we use six workloads. The six workloads are divided into three categories. The first category is a CPU intensive type which includes kernel and apache compilation workloads. The kernel compile workload compiles Linux 2.6.38 vanilla kernel. The Apache compile workload compiles the Apache web server 2.219 version. The next category is a memory intensive type. The SPECjbb 2005 is a typical memory intensive workload evaluating performance of a server side Java program. Another workload is VolanoMark [32] which simulates thousands of chat clients. For evaluating I/O performance, we use SPECweb 2005 and RUBiS. SPECweb 2005 evaluates web servers and RUBiS simulates an auction site. For the two compiler workloads, the inverse of elapsed time is used as the performance metric, and for SPECjbb 2005, we use the performance score as the performance metric. For SPECweb 2005, VolanoMark, and RUBiS, the average throughput is used as the metric.

7.2 Performance of Workloads

SMM operation. First, we present the execution for each individual H-SVM operation. Table 1 summarizes the elapsed time for the operations. Null operation is used to evaluate the overhead of SMM switching without any processing. Null takes about 5 μ s, showing that H-SVM_{smm} has a modest overhead of SMM switching for each SMM operation. Most of operations take between 300 and 600 μ s. Page unmap operation takes about 200 μ s longer than page map, to clean up the page content. The operations denoted by 512

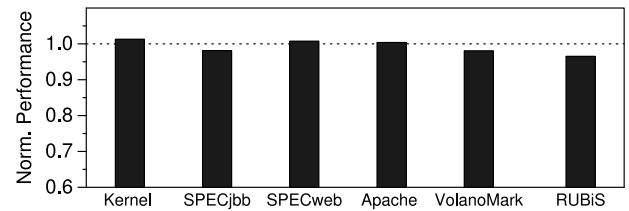


Fig. 8. Performance without ballooning, normalized to the system without H-SVM.

shows the 2 MB super paging effect, which maps 512 4KB pages together for each page map operation.

VM creation. Second, we evaluate the performance impact of page map operations for a VM creation. Fig. 7 shows the total elapsed times for VM creation with H-SVM_{smm}. We use four VM types with memory sizes from 512 MB to 4 GB. The VM creation time is the time interval from the initiation of a VM creation to the completion of VM construction. It does not include the time to actually boot the guest OS on the constructed VM. In the VM creation without H-SVM_{smm}, the elapsed times only slightly increase with larger memory sizes. However, the VM creation times with H-SVM_{smm} increase as the memory sizes increase, due to the extra overhead of SMM invocation. A VM creation operation with H-SVM_{smm} takes 2-5 sec longer than one without H-SVM_{smm}.

Real workloads without ballooning. Fig. 8 shows the performance impact of realistic workloads without dynamic memory management. The performance is normalized to the runs without H-SVM_{smm}. After the VM creation, the hypervisor rarely changes nested page tables, since dynamic memory management with ballooning is not enabled in the experiment. Disabling dynamic memory management is not uncommon in real cloud systems, since the service level agreement must always guarantee the memory capacity for user VM. The figure shows that the performance changes are negligible in all the results within possible error ranges, as the performance slightly improves or degrades in random patterns. Without dynamic memory management by the hypervisor, H-SVM_{smm} does not affect the overall performance significantly.

7.3 Dynamic Memory Mapping Changes

To evaluate the effect of dynamic memory allocation for virtual machines, we evaluate the effect of the balloon driver for each VM. To increase dynamic memory allocation and deallocation rates, the balloon driver is configured to aggressively free any unused memory pages to a pool of free pages every five seconds. All guest VMs share the pool of free pages, and if a VM requires more physical memory

TABLE 1
Performance of SMM Operations

Operation	Null	Create VM	Delete VM	Page map	Page map (512)	Page unmap	Page unmap (512)
AVG (μ s)	5.38	349	325	341	716	536	104,286
Min (μ s)	4.83	323	318	338	699	529	98,527
Max (μ s)	6.30	414	348	348	732	573	111,754
STD (μ s)	0.36	18.8	5.00	1.65	7.27	8.92	3,139

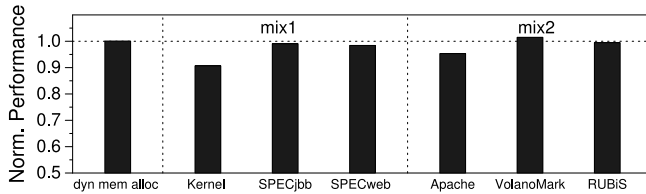


Fig. 9. Performance with memory ballooning, normalized to the system without H-SVM.

pages, the hypervisor allows the requesting VM to use free pages in the pool. The unit of ballooning is a 2 MB large page to reduce SMI invocations. Each guest VM is created with 4 GB memory initially, but the actual allocated physical memory for a VM changes dynamically.

A microbenchmark, *dyn_mem_alloc*, repeats the allocation and deallocation of 1 GB memory 20 times. It allocates and touches a 1 GB memory region, and deallocates the memory. The allocation and deallocation are repeated after a 10 second sleep period. During the sleep period, the balloon driver in the guest VM will deallocate unused memory pages to the free page pool. For *dyn_mem_alloc*, the total elapsed time is used as the performance metric. In addition to the microbenchmark, we use two realistic mixed workloads, each of which runs three applications. The *mix1* workload consists of kernel compile, SPECjbb 2005, and SPECweb 2005 workloads. The *mix2* workload consists of Apache compile, VolanoMark [32] and RUBiS [23].

Fig. 10 presents the number of page map and unmap operations during the execution of *dyn_mem_alloc*, *mix1*, and *mix2*. The solid lines are the numbers of page map operations, and the dotted lines are the numbers of page unmap operations. For all three cases, there are some page map and unmap operations soon after the VM creations (50-200 seconds). It is because the balloon driver on each VM with 4 GB memory releases the unused memory. For *mix1* and *mix2*, there are high rates of page map operations between 300-400 seconds, when the applications on VMs start requesting more pages. For *dyn_mem_alloc*, the memory mappings for VMs change very dynamically, as we designed the microbenchmark to generate high rates of memory mapping changes.

However, the performance impacts by page map and unmap operations are small for memory and I/O intensive workloads. Fig. 9 presents the performance of each application for the three workloads. CPU intensive workloads such as kernel and apache compilation have some overheads due to the synchronized SMM invocation as mentioned Section 6.5. When page map and unmap operations occur, all of the cores enter SMM, which causes a significant negative impact on such CPU-sensitive tasks.

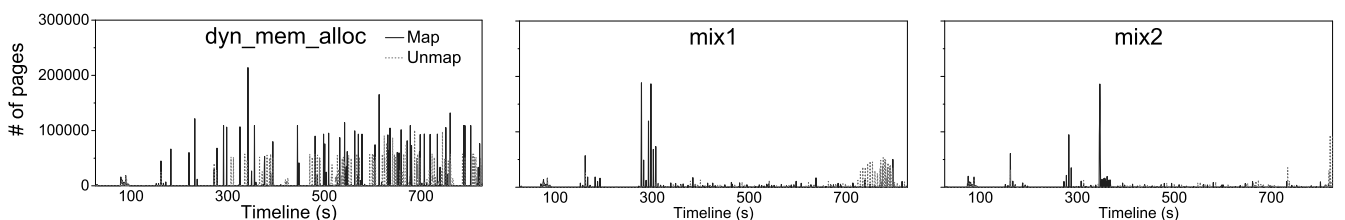


Fig. 10. Page map and unmap operations during the execution of workloads.

TABLE 2
Complexity of H-SVM_{smm}

Component	Type	LOC (Lines of Code)
SMM initialization	Assembly	221
SMRAM Management	C language	166
VM Handling	C language	88
NPT Handling	C language	376
OST Handling	C language	152
Red-Black Tree	C language	306
Utility	C language	120
SUM		1,429

7.4 Complexity of H-SVM_{smm}

The complexity of H-SVM_{smm} is relatively low with modest changes of the SMI handler. Table 2 shows the complexity of H-SVM_{smm} by measuring the additional lines of code for the SMI handler. The total number of lines is less than 1,500 lines, which shows a low complexity of H-SVM_{smm}.

8 RELATED WORK

This paper extends our prior studies of hardware-based secure virtualization with support for content-based page sharing, solutions to availability problems, and discussions on integrity supports [15].

There have been many prior studies to embed security features into hardware processors, under the assumption that compromising hardware components is harder than attacking software systems. Such hardware attacks often require physical accesses to systems as well as more sophisticated attacking methods than remote software attacks. Prior studies for hardware-oriented protection reduce TCB to the hardware system without software stacks, or with a thin verifiable software layer [16], [18], [27].

The Execution-Only Memory (XOM) architecture provides the integrity and privacy of applications with a hardware mechanism, supporting copy and tamper-resistance [18]. A subsequent work extends the supports for copy and tamper-resistance for applications even under an untrusted OS [17]. AEGIS supports tamper-evident and tamper-resistant environments using a combination of secure kernel and hardware mechanisms [27]. It proposes a broad range of solutions, assuming either a trusted or untrusted kernel. Both XOM and AEGIS can reduce TCB to the hardware processor and assume that all the other system components, such as external memory, are vulnerable. To provide solutions for attacks to the hardware components other than the processor, the systems use sophisticated hardware-based integrity checking and encryption mechanisms.

Recently, there have been several studies for software-based VM memory protection mechanisms, relying on trusted hypervisors. In such approaches, the hypervisor must be included in TCB. Overshadow uses memory shadowing to protect sensitive applications from a malicious OS [8]. When an application is running, the hypervisor provides the application with a normal memory view. However, if the kernel is running, the hypervisor provides the kernel with a limited memory view for applications, which shows only the encrypted application data. SP3 uses a similar memory shadowing mechanism, but SP3 determines the memory view according to the access permission set to each memory page [37].

Using virtualization techniques, systems can be divided into trusted and untrusted compartments. The Terra system supports closed-box virtual machines called trusted execution environments [11]. It verifies the components of virtual machines such as the boot loader, kernel, and sensitive applications using a trusted platform module at the start-up time. Proxos is similar to the Terra system in providing isolated and trusted computing environments [29]. While Terra does not allow the use of commodity OS resources for a trusted execution environment, Proxos supports the use of commodity OS resources by partitioning system call interfaces and routing system calls from a trusted VM to a normal VM.

Several recent studies discuss the vulnerability of hypervisors, and propose solutions to improve their security. To prevent control-flow hijacking attacks to hypervisors, Hypersafe assures the control-flow integrity of a running hypervisor [34]. HyperSentry measures the integrity of a running hypervisor in a stealthy way to prevent the compromised hypervisor from disabling the integrity checking component [7]. In the Xen hypervisor, the management domain (domain0) often becomes the target of attacks, as it has a higher privileged functionality than guest VMs, but has a similar vulnerability. Gordon divides the management domain into trusted and untrusted components to minimize TCB [21]. Li and Jha proposed a secure VM execution environment to protect guest virtual machines from the untrusted management domain [9]. With this scheme, the management domain cannot access any memory of a virtual machine after the creation of the virtual machine.

CloudVisor adds an extra software layer under a conventional hypervisor, to protect virtual machines from a compromised hypervisor [10]. This approach is similar to our proposed system, as both systems attempt to separate critical functions of updating security-sensitive data structures from the hypervisor. However, CloudVisor uses a software-based approach to add an extra layer using the current hardware support for virtualization. Hypercoffer [36] provides a hardware-software framework that protects confidentiality and integrity of VMs even under physical attacks. The TCB of Hypercoffer includes only processor chips, and HyperCoffer supports memory encryption and integrity checking to off-chip memory.

9 CONCLUSIONS

In this paper, we proposed a hardware-based mechanism called H-SVM to isolate the memory of a VM securely even

under a compromised hypervisor. Unlike prior hardware-based mechanisms which support tamper-evidence or tamper-resistance for a hardware attack, H-SVM simplifies the complexity of hardware supports significantly by assuming a software-only threat model. We believe this restricted threat model is appropriate in the current cloud computing environments, where systems can be protected from physical intrusions in an isolated data center. Based on the H-SVM design, we implemented a prototype system using system management mode, proving the low complexity and feasibility of H-SVM.

ACKNOWLEDGMENTS

This work was partly supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIP) (No. NRF-2010-0025511, No. 2012R1A1A1014586, and No. NRF-2013R1A2A2A01015514), and by the IT R&D program of MSIP/IITP [10041313, UX-oriented Mobile SW Platform]. J. Huh is the corresponding author.

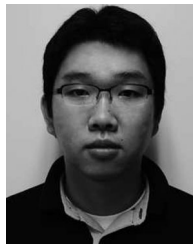
REFERENCES

- [1] Advanced Micro Devices, AMD I/O Virtualization Technology (IOMMU) Specification, 2009.
- [2] Advanced Micro Devices, Secure Virtual Machines Architecture Reference Manual, 2005.
- [3] Advanced Micro Devices. AMD64 Architecture Programmer's Manual: Volume 2: System Programming, 2007.
- [4] Advanced Micro Devices. AMD-V Nested Paging, 2008.
- [5] Amazon Elastic Compute Cloud (Amazon EC2). (2008) [Online]. Available: <http://aws.amazon.com/ec2>
- [6] R. Anderson and M. Kuhn, "Low cost attacks on tamper resistant devices," in *Proc. Secur. Protocols: 5th Int. Workshop*, 1997, pp. 125–136.
- [7] A. M. Azab, P. Ning, Z. Wang, X. Jiang, X. Zhang, and N. C. Skalsky, "HyperSentry: Enabling stealthy in-context measurement of hypervisor integrity," in *Proc. 17th ACM Conf. Comput. Commun. Secur.*, 2010, pp. 38–49.
- [8] X. Chen, T. Garfinkel, E. C. Lewis, P. Subrahmanyam, C. A. Waldspurger, D. Boneh, J. Dwoskin, and D. R. K. Ports, "Overshadow: A virtualization-based approach to retrofitting protection in commodity operating systems," in *Proc. 13th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2008, pp. 2–13.
- [9] A. R. Chunxiao Li and N. K. Jha, "Secure virtual machine execution under an untrusted management OS," in *Proc. IEEE 3rd Int. Conf. Cloud Comput.*, 2010, pp. 172–179.
- [10] H. Chen, F. Zhang, Jin Chen, and B. Zang, "CloudVisor: Retrofitting protection of virtual machines in multi-tenant cloud with nested virtualization," in *Proc. 23rd ACM Symp. Oper. Syst. Principles*, 2011, pp. 203–216.
- [11] T. Garfinkel, B. Pfaff, J. Chow, M. Rosenblum, and D. Boneh, "Terra: A virtual machine-based platform for trusted computing," in *Proc. 19th ACM Symp. Oper. Syst. Principles*, 2003, pp. 193–206.
- [12] D. Gupta, S. Lee, M. Vrabie, S. Savage, A. C. Snoeren, G. Varghese, G. M. Voelker, and A. Vahdat, "Difference engine: Harnessing memory redundancy in virtual machines," in *Proc. 8th USENIX Conf. Oper. Syst. Des. Implementation*, 2008, pp. 309–322.
- [13] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Calandrino, A. J. Feldman, J. Appelbaum, and E. W. Felten, "Lest we remember: Cold-boot attacks on encryption keys," *Commun. ACM*, vol. 52, pp. 91–98, May 2009.
- [14] Intel, "Intel virtualization technology for directed I/O Architecture Specification," Number D51397-007, Rev. 2.3, 2014.
- [15] S. Jin, J. Ahn, S. Cha, and J. Huh, "Architectural support for secure virtualization under a vulnerable hypervisor," in *Proc. 44th Annu. IEEE/ACM Int. Symp. Microarchit.*, 2011, pp. 272–283.
- [16] R. B. Lee, P. C. S. Kwan, J. P. McGregor, J. Dwoskin, and Z. Wang, "Architecture for protecting critical secrets in microprocessors," in *Proc. 32nd Annu. Int. Symp. Comput. Archit.*, 2005, pp. 2–13.

- [17] D. Lie, C. A. Thekkath, and M. Horowitz, "Implementing an untrusted operating system on trusted hardware," in *Proc. 19th ACM Symp. Oper. Syst. Principles*, 2003, pp. 178–192.
- [18] D. Lie, C. A. Thekkath, M. Mitchell, P. Lincoln, D. Boneh, J. C. Mitchell, and M. Horowitz, "Architectural support for copy and tamper resistant software," in *Proc. 9th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2000, pp. 168–177.
- [19] R. C. Merkle, "Protocols for public key cryptosystems," in *Proc. IEEE Symp. Secur. Privacy*, 1980, pp. 122–134.
- [20] G. Milos, D. G. Murray, S. Hand, and M. A. Fetterman, "Satori: Enlightened page sharing," in *Proc. Conf. USENIX Annu. Tech. Conf.*, 2009, pp. 1–14.
- [21] D. G. Murray, G. Milos, and S. Hand, "Improving xen security through disaggregation," in *Proc. 8th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, 2008, pp. 151–160.
- [22] G. Neiger, A. Santoni, F. Leung, D. Rodger, and R. Uhlig, "Intel virtualization technology: Hardware support for efficient processor virtualization," *Intel Technol. J.*, vol. 10, no. 03, pp. 167–178, 2006.
- [23] RUBiS Benchmark. (2008) [Online]. Available: <http://rubis.ow2.org>
- [24] Secunia vulnerability report: VMware ESX Server 4.x. (2010) [Online]. Available: <http://secunia.com/advisories/product/25985/>
- [25] Secunia vulnerability report: Xen 3.x. (2010) [Online]. Available: <http://secunia.com/advisories/product/15863/>
- [26] Security is chief obstacle to cloud computing adoption. (2009) [Online]. Available: <http://www.darkreading.com/securityservices/security/perimeter/showArticle.jhtml?articleID=221901195>
- [27] G. E. Suh, D. Clarke, B. Gassend, M. van Dijk, and S. Devadas, "AEGIS: Architecture for tamper-evident and tamper-resistant processing," in *Proc. Int. Conf. Supercomput.*, 2003, pp. 160–171.
- [28] Survey: Cloud computing "No Hype," but fear of security and cloud slowing adoption. (2009) [Online]. Available: http://www.circleid.com/posts/20090226_cloud_computing_hype_security
- [29] R. Ta-Min, L. Litty, and D. Lie, "Splitting interfaces: Making trust between applications and operating systems configurable," in *Proc. 7th Symp. Oper. Syst. Des. Implementation*, 2006, pp. 279–292.
- [30] Trusted platform module. [Online]. Available: http://www.trustedcomputinggroup.org/developers/trusted_platform_module, 2005.
- [31] VMware ESX and ESXi. (2010) [Online]. Available: <http://www.vmware.com/products/vsphere/esxi-and-esx/index.html>
- [32] VolanoMark. (2009) [Online]. Available: <http://www.volano.com/benchmark>
- [33] C. A. Waldspurger, "Memory resource management in vmware esx server," in *Proc. 5th Symp. Oper. Syst. Des. Implementation*, 2002, pp. 181–194.
- [34] Z. Wang and X. Jiang, "HyperSafe: A lightweight approach to provide lifetime hypervisor control-flow integrity," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 380–395.
- [35] Windows Azure Platform. (2010) [Online]. Available: <http://www.microsoft.com/windowsazure/>
- [36] Y. Xia, Y. Liu, and H. Chen, "Architecture support for guest-transparent VM protection from untrusted hypervisor and physical attacks," in *Proc. IEEE 19th Int. Symp. High Perform. Comput. Archit.*, 2013, pp. 246–257.
- [37] J. Yang and K. G. Shin, "Using hypervisor to provide data secrecy for user applications on a per-page basis," in *Proc. 4th Int. Conf. Virtual Execution Environ.*, 2008, pp. 71–80.



Seongwook Jin received the BS degree in computer science and engineering from Kumoh National University in 2008 and the MS degree in computer science from KAIST in 2010. He is currently working towards the PhD degree in computer science at KAIST. His research interests include computer architecture, security, and virtualization.



Jeongseob Ahn received the BS degree in computer science and engineering from Dongguk University in 2009 and the MS degree in computer science from KAIST in 2011. He is currently working towards the PhD degree in computer science at KAIST. His research interests include computer architecture, operating systems, and virtualization.



Jinho Seol received the BS degree in computer science and engineering from Chung-Ang University, the MS degree in computer science from KAIST in 2009, and is working towards the PhD degree in computer science at the Korea Advanced Institute of Science and Technology (KAIST). His research interests include system security, virtualization, and computer architecture. He worked for Jiransoft as a software engineer from 2000 to 2003.



Sanghoon Cha received the BS degree in computer from Hanyang University, Seoul, Korea in 2010. He is currently working towards the MS degree in computer science from the Korea Institute of Science and Technology (KAIST), Daejeon, Korea. His research interests include computer architecture, operating system, cloud computing, and virtualization.



Jaehyuk Huh received the BS degree in computer science from Seoul National University, and the MS and PhD degree in computer science from the University of Texas at Austin. He is an associate professor of computer science at the Korea Advanced Institute of Science and Technology (KAIST). His research interests include computer architecture, parallel computing, virtualization, and system security.



Seungryoul Maeng received the BS degree in electronics engineering from Seoul National University (SNU), Republic of Korea, in 1977, and the MS and PhD degrees in computer science from KAIST, in 1979 and 1984, respectively. He is a professor of computer science at the Korea Advanced Institute of Science and Technology (KAIST) since 1984. His research interests include micro-architecture, parallel processing, cluster computing, and embedded systems. From 1988 to 1989, he was with the University of Pennsylvania as a visiting scholar.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.